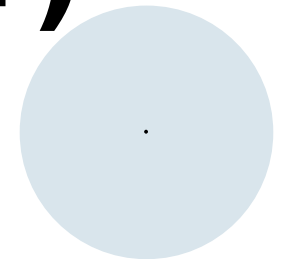


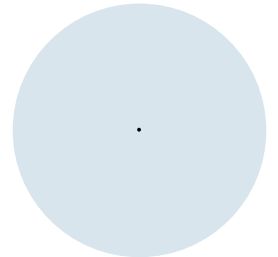
Einführung in die Programmierung (WIN+BIT)

I/O, Kontrollstrukturen - Fallunterscheidungen

Prof. Dr. Johannes C. Schneider



Ein- und Ausgabe auf der Konsole



Java main()

- Need JDK 25
- New File -> Main.java

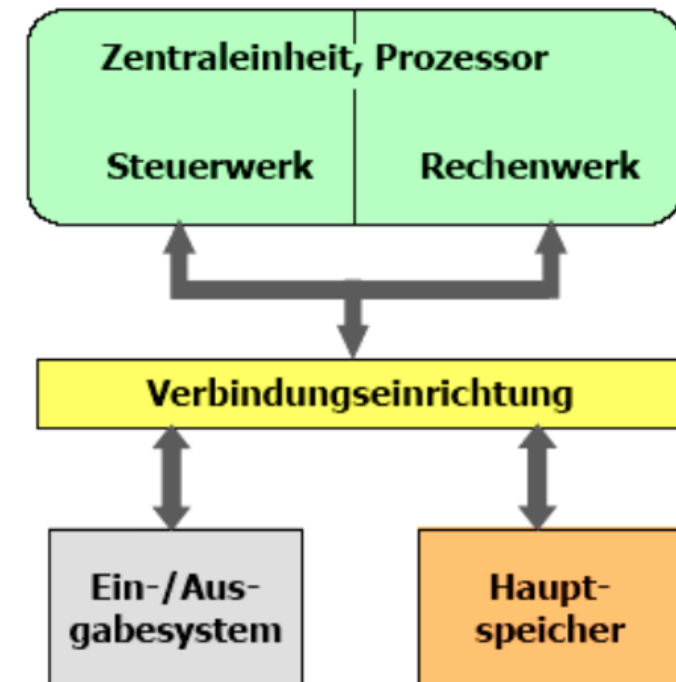
```
void main() {  
    System.out.println("Hello World!");  
}
```

4. Grundlagen der Rechnerarchitektur

Komponenten eines von Neumann Rechners

▪ Ein-/ Ausgabesystem (Peripheriegeräte)

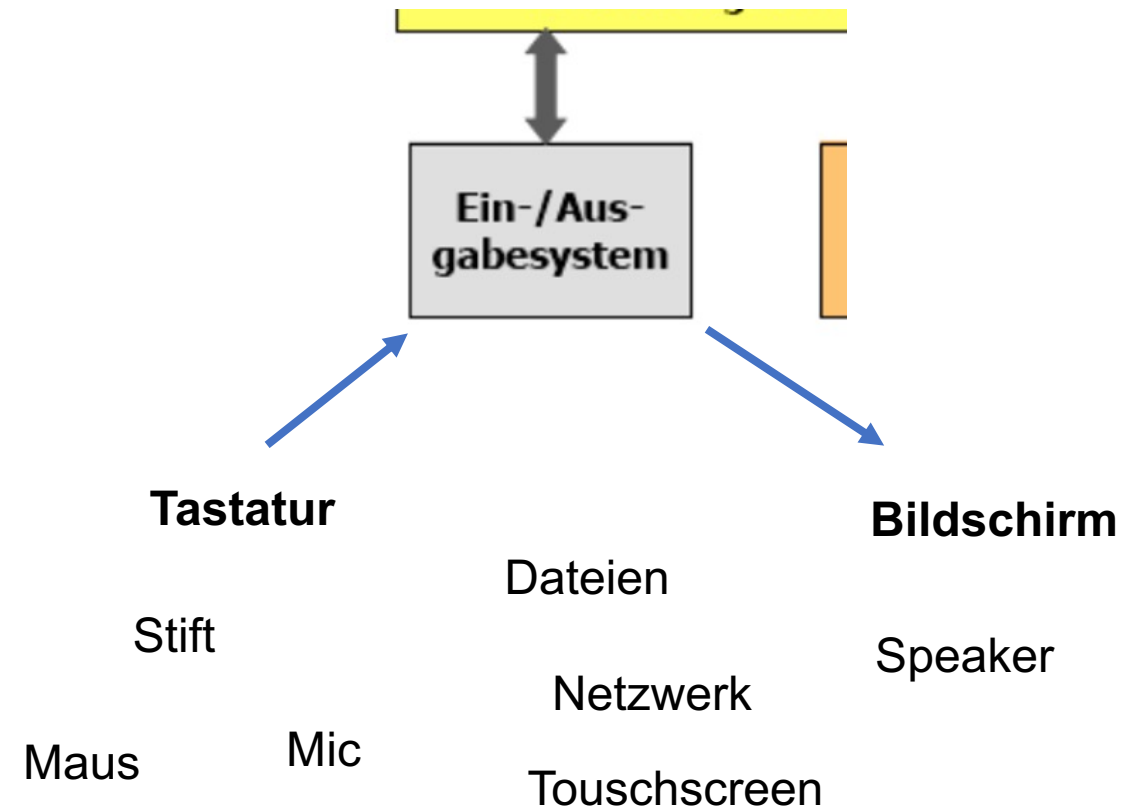
- Geräte zur Eingabe von Daten und Programmen und zur Ausgabe der verarbeiteten Daten (Bildschirm, Drucker, Terminals, ...)
- Diese Geräte sind über Ein-/Ausgabe-Schnittstellen mit dem Rechner verbunden.
- Die Verbindung der Schnittstellen mit dem Prozessor (und zu den Peripheriegeräten) geschieht durch Adress-, Daten- und Steuerleitung.



Ein-/Ausgabe in Java

Standard In / Standard Out

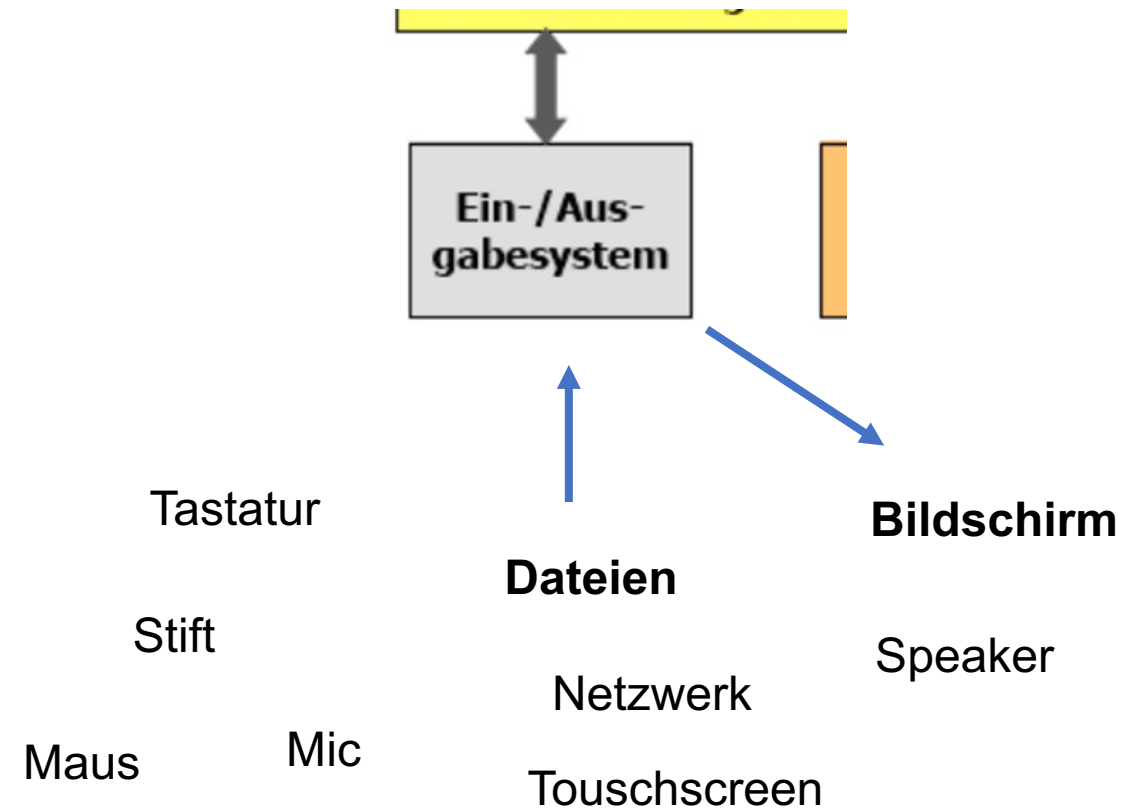
- Jedes Programm hat einen Standard-Eingabe Kanal und einen Standard Ausgabe Kanal für Text Eingabe / Ausgabe.
 - Standard-In (**System.in**) verweist auf das Peripheriegerät, dass als **Eingabegerät** angenommen wird.
Dies ist typischerweise die Tastatur
 - Standard-Out (**System.out**) verweist auf das Peripheriegerät, dass als **Ausgabegerät** angenommen wird.
Dies ist typischerweise ein Textfenster (Console) auf dem Bildschirm.



Ein-/Ausgabe in Java

Standard In / Standard Out

- Die Standardeingabe kann auch einer anderen Eingabequelle zugeordnet werden, z.B. **einer Datei**.



BildschirmAusgabe

- `System.out.println(<Ausgabe>)` – Ausgabe mit Zeilenumbruch
- `System.out.print(<Ausgabe>)` – Ausgabe ohne Zeilenumbruch
- Ausgabe kann ein String sein, aber auch ein primitiver Datentyp.

BildschirmAusgabe, formatiert

- `System.out.printf(<Format mit Platzhaltern>, Parameter1, Parameter2, ...)`
→ Ausgabe der Parameter ohne Zeilenumbruch in angegebenem Format
 - Platzhalter werden der Reihe nach ersetzt:
 - `%d` für Ganzzahl-Parameter (byte, short, int float)
 - `%f` für Gleitkommazahl-Parameter (float, double)
 - `%s` für String-Parameter
 - `%n` für neue Zeile
 - `String.format()` funktioniert genauso, liefert aber einen formatierten String zurück
- <http://docs.oracle.com/javase/tutorial/essential/io/formatting.html>



Nützliche Formatierungen

%.2f	zwei Nachkommastellen (gerundet), auch andere Werte möglich	2.3456 → 2.35
%10s	Auffüllen auf Breite 10	"Anna" → "____Anna" *
%10.2f	Kombination von beidem	"1.234" → "____1.23" *
%+d	Vorzeichen einer Zahl erzwingen	12 → +12 -12 → -12
%,d	Tausender-Trenner	12345 → 12.345 (deutsches System)

* Hier stehen jeweils 6 Leerzeichen



Tastatureingabe

```
import java.util.Scanner;
...
Scanner scanner = new Scanner(System.in);
...
String text = scanner.next();
int number = scanner.nextInt();
...
// Am Ende der Methode
scanner.close();
```

- Scanner ist im Package `java.util`
- Import macht Scanner in der Klasse verfügbar
- Alternativ über vollständigen Namen referenzieren:
`Scanner = new java.util.Scanner(System.in);`
- Siehe auch [Scanner \(Java API\)](#)



Scanner, nächster Wert

<code>String</code>	<code>next()</code>
<code>boolean</code>	<code>nextBoolean()</code>
<code>byte</code>	<code>nextByte()</code>
<code>byte</code>	<code>nextByte(int radix)</code>
<code>double</code>	<code>nextDouble()</code>
<code>float</code>	<code>nextFloat()</code>
<code>int</code>	<code>nextInt()</code>
<code>int</code>	<code>nextInt(int radix)</code>
<code>String</code>	<code>nextLine()</code>
<code>long</code>	<code>nextLong()</code>
<code>long</code>	<code>nextLong(int radix)</code>
<code>short</code>	<code>nextShort()</code>
<code>short</code>	<code>nextShort(int radix)</code>

- Siehe auch [Scanner \(Java API\)](#)

Die ausgegrauten Funktionen müssen Sie nicht kennen

Erklärung:

- radix: Basis der Zahldarstellung
- z.B. radix=16 zum Parsen von Hexadezimalzahlen

Scanner-Konfiguration

- Locale: Gebietsschema, wie z.B. auch Zahlenformat (1,000.42 US versus 1.000,42 DE), siehe folgende Folie
- Delimiter: Zeichen anhand dessen die Eingabe zerlegt wird.
 - Default: "Whitespace", also Leerzeichen, Tab, Enter, ...

```
import java.util.Locale;
import java.util.Scanner;
...
Scanner scanner = new Scanner(System.in);
scanner.useLocale(Locale.US);
scanner.useDelimiter("\n");
...
```

Locale

"Das **Locale** ist ein Einstellungssatz, der die **Gebietsschemaparameter** (Standortparameter) für Computerprogramme enthält.

Dazu gehören in erster Linie die **Sprache der Benutzeroberfläche, das Land und Einstellungen zu Zeichensatz, Tastaturlayout, Zahlen-, Währungs-, Datums- und Zeitformaten**. Ein Einstellungssatz wird üblicherweise mit einem Code, der meist Sprache und Land umfasst, eindeutig identifiziert."

(Q: Wikipedia)

Beispiele für diese Codes:

- en_US: englisch, United States
- en_GB: englisch, Großbritannien
- de_DE: deutsch, Deutschland
- de_CH: deutsch, Schweiz

Locale für formatierte Ausgabe

- In Java sind diese Konstanten als
 - `java.util.Locale.GERMANY` (`de_DE`)
 - `java.util.Locale.UK` (`en_GB`)
 - ...verfügbar

- Locale beeinflusst auch `printf`/`format`:

Locale kann man explizit als ersten Parameter mitgeben

```
System.out.printf("%.4f%n", Math.PI); // 3,1416 auf deutschem System
System.out.printf(Locale.ENGLISH, "%.4f%n", Math.PI); // 3.1416
String piWithComma = String.format(Locale.GERMANY, "%.4f", Math.PI); // 3,1416
```

Pro-Tipp: `String.format` funktioniert ähnlich wie `printf`, gibt aber den Wert nicht aus, sondern liefert ihn zurück.
(In `piWithComma` steht danach die Zahl π auf 4 Stellen gerundet.)



Beispiel: CertificatePrinter

- Wir wollen Name, Alter und Note einlesen (mit Scanner).
- Anschließend geben wir ein schön formatiertes Zertifikat aus.

```
Bitte Name eingeben: Beate Beispiel
Bitte Alter eingeben: 21
Bitte Note eingeben: 1,3
=====
EPR-Scanner-Zertifikat

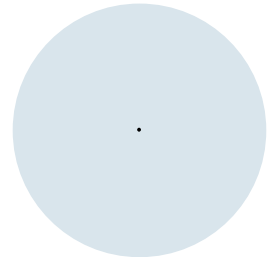
Hiermit wird
-= Beate Beispiel (Alter 21) -=
bestätigt, dass er/sie den Umgang mit Scanner
-= mit der Note 1,3 -=
beherrscht.
=====
```



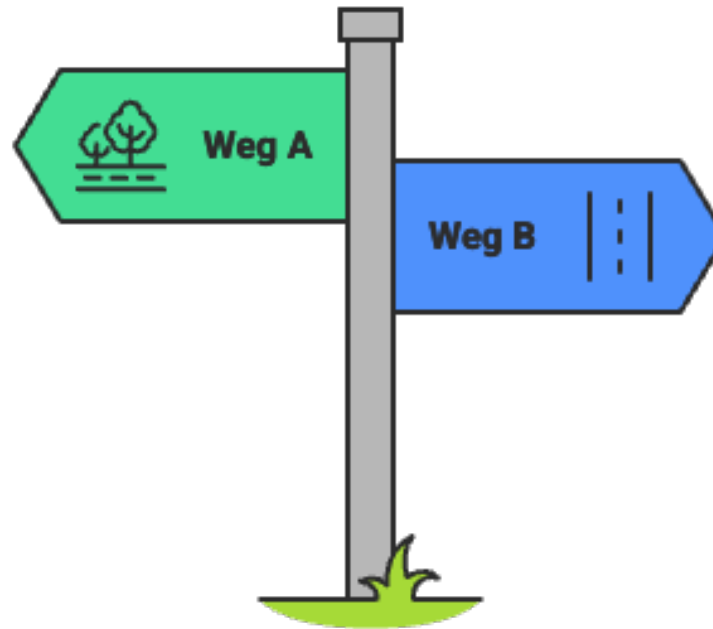
code04.

Certificate
Printer

Kontrollstrukturen



Welchen Weg soll ich wählen?



Made with  Napkin

Eintrittspreis Schwacketenbad

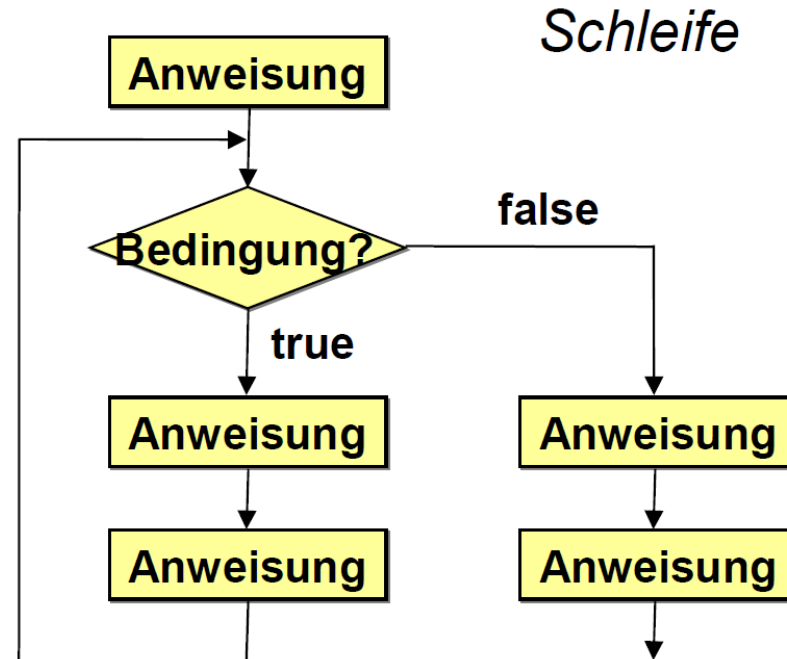
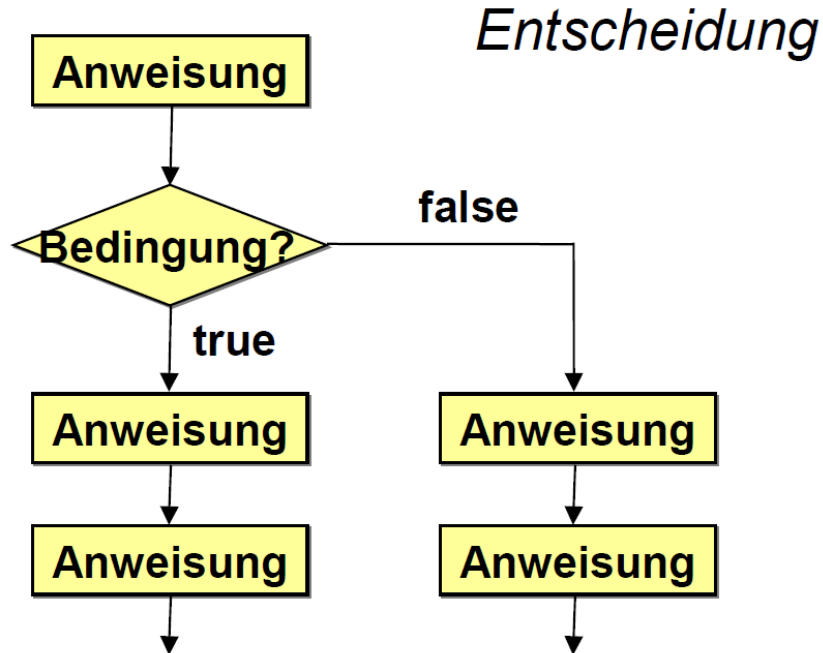
Kategorie	Preis Montag - Freitag	Preis Wochenende & Feiertage
Erwachsene	6,50 €	7,50 €
Ermäßigte*	5,00 €	6,00 €

- Eingabe ?
- Verarbeitung?
- Ausgabe=?



Was sind Kontrollstrukturen?

- Kontrollstrukturen erlauben es, den Ablauf eines Programms zu verändern (statt Zeile für Zeile).
- In Abhängigkeit von Bedingungen können bestimmte Programmteile ausgeführt werden.



Anweisungen

- Ein Programm besteht aus einer Folge von Anweisungen, **Kontrollstrukturen** sind **spezielle Anweisungen**
- Sie kennen schon Deklarations-Anweisungen, Zuweisungs-Anweisungen, (Methoden-Aufrufe)
- Heute:
 - Blöcke
 - if-Anweisungen
 - switch-Anweisungen

Blöcke und lokale Variablen

- Ein Block ist eine **geklammerte** `{}` Folge von Anweisungen
- **Blöcke** können immer **statt einzelner Anweisungen** eingesetzt werden
- **lokale Variablen** sind Variablen, die innerhalb eines Blocks definiert werden
 - Wir haben es bisher nur mit lokalen Variablen zu tun gehabt (Variablen die innerhalb des main-Methoden-Blocks deklariert wurden)

Blöcke

Block
Block

```
int outerNumber = 1;
{
    int innerNumber = 2;
    System.out.println(outerNumber+innerNumber);
}
{
    int innerNumber = 3;
    System.out.println(outerNumber+innerNumber);
}
```

- lokale Variablen sind nur innerhalb des Blocks, wo sie definiert werden, gültig und zugreifbar
- Innerhalb eines Block kann man auf Variablen des umgebenden Blocks zugreifen (outerNumber)



Blöcke

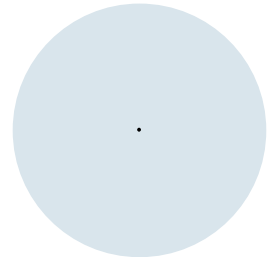
Compilerfehler!

```
int outerNumber = 1;
{
    int innerNumber = 2;
    int outerNumber = 5;
    System.out.println(outerNumber+innerNumber);
}
innerNumber = 5;
```

- Innerhalb eines Block kann man Variablen des umgebenden Blocks nicht neu definieren
- Außerhalb eines Block kann man nicht auf lokale Variablen des Blocks zugreifen



Fallunterscheidungen I

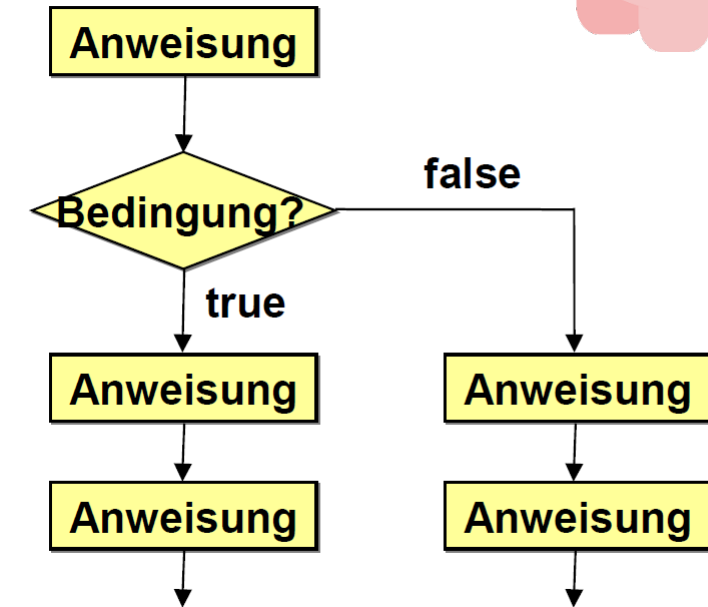


if-Anweisung

- Bedeutendste aller Kontrollanweisungen, Verzweigung
- Erlaubt selektive Ausführung von Programmteilen

```
if (<Bedingung>) {  
    <Anweisung(en)> // Falls Bedingung true  
} else {  
    <Anweisung(en)> // Falls Bedingung false  
}
```

- <Bedingung> ist Ausdruck vom Typ boolean
- Auswertung:
Falls <Bedingung> true, dann unmittelbar folgende Anweisung, sonst Anweisung nach else.



if-Anweisung

– Beispiel Betrags-Berechnung

```
double x, y;  
java.util.Scanner scanner =  
    new java.util.Scanner(System.in);  
System.out.print("x: ");  
x = scanner.nextDouble();  
  
if (x < 0.0) {  
    y = -x;  
} else {  
    y = x;  
}  
System.out.println("Der Betrag von " + x + " ist " + y);
```



code04.
Absolute
Value1

if-Anweisung

– Beispiel Betrags-Berechnung

```
double x, y;  
java.util.Scanner scanner =  
    new java.util.Scanner(System.in);  
System.out.print("x: ");  
x = scanner.nextDouble();
```

```
if (x < 0.0) {  
    y = -x;  
} else {  
    y = x;  
}
```

```
y = x;  
if (x < 0.0) {  
    y = -x;  
}
```

Alternative ohne else

```
System.out.println("Der Betrag von " + x + " ist " + y);
```



code04.
Absolute
Value2

if-Anweisung

- Beispiel Betrags-Berechnung

```
double x, y;  
java.util.Scanner scanner =  
    new java.util.Scanner(System.in);  
System.out.print("x: ");  
x = scanner.nextDouble();
```

```
if (x < 0.0) {  
    y = -x;  
} else {  
    y = x;  
}
```

```
y = (x < 0.0) ? -x : x;
```

**Alternative mit Bedingungsoperator
(falls Anweisung einfache Zuweisung)**

```
System.out.println("Der Betrag von " + x + " ist " + y);
```



code04.
Absolute
Value3

Beispiel Altersprüfung

- Beispiel von der letzten Einheit (Überprüfung der Volljährigkeit)

```
if (alter >= 18) {  
    System.out.println("Du bist volljährig.");  
} else {  
    System.out.println("Du bist noch nicht volljährig.");  
}
```



if-Anweisung, Blöcke

```
if (<Bedingung>) {  
    <Anweisung(en)>    // Block 1  
} else {  
    <Anweisung(en)>    // Block 2  
}
```

- Eine if-Anweisung beinhaltet zwei Blöcke
 - es können lokale Variablen definiert werden
 - es kann auf äußere Variablen zugegriffen werden

if-Anweisungen immer klammern!

- Eine **einzelne** Anweisung **muss nicht geklammert** werden, also z.B.
 - if (<Bedingung>) <Anweisung>;
 - if (<Bedingung>) <Anweisung> else <Anweisung>;

– **Fehlerquelle:**

```
Scanner scanner = new Scanner(System.in);
System.out.println("Geben Sie eine ganze Zahl ein:");
int x = scanner.nextInt();
if (x == 0)
    System.out.println("x ist gleich 0");
else
    System.out.println("x ungleich 0, wir dividieren");
    System.out.println("1/x liefert " + 1.0 / x);
```

nicht mehr
innerhalb von
else!

- Vorbeugen!
 - Auch einzelne Anweisungen klammern {}
 - Quellcode-Formatierung durch IDE
(Strg + Alt + L in IntelliJ)



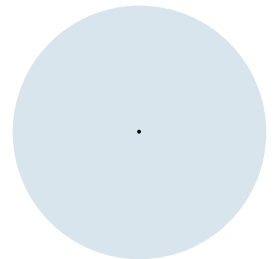
code04.
Brackets

if-Anweisungen immer klammern!

- **Vorbeugen!**
 - Auch einzelne Anweisungen klammern {}
 - Quellcode-Formatierung durch IDE
(Strg + Alt + L in IntelliJ)

```
Scanner scanner = new Scanner(System.in);
System.out.println("Geben Sie eine ganze Zahl ein:");
int x = scanner.nextInt();
if (x == 0) {
    System.out.println("x ist gleich 0");
} else {
    System.out.println("x ungleich 0, wir dividieren");
    System.out.println("1/x liefert " + 1.0 / x);
}
```


Einschub: Debugging



"Printf-Debugging"

```
if (x < 0.0) {  
    System.out.printf("Fall x < 0 mit x: %f%n", x);  
    y = -x;  
} else {  
    System.out.printf("Fall else mit x: %f%n", x);  
    y = x;  
}  
System.out.println("Der Betrag von " + x + " ist " + y);
```

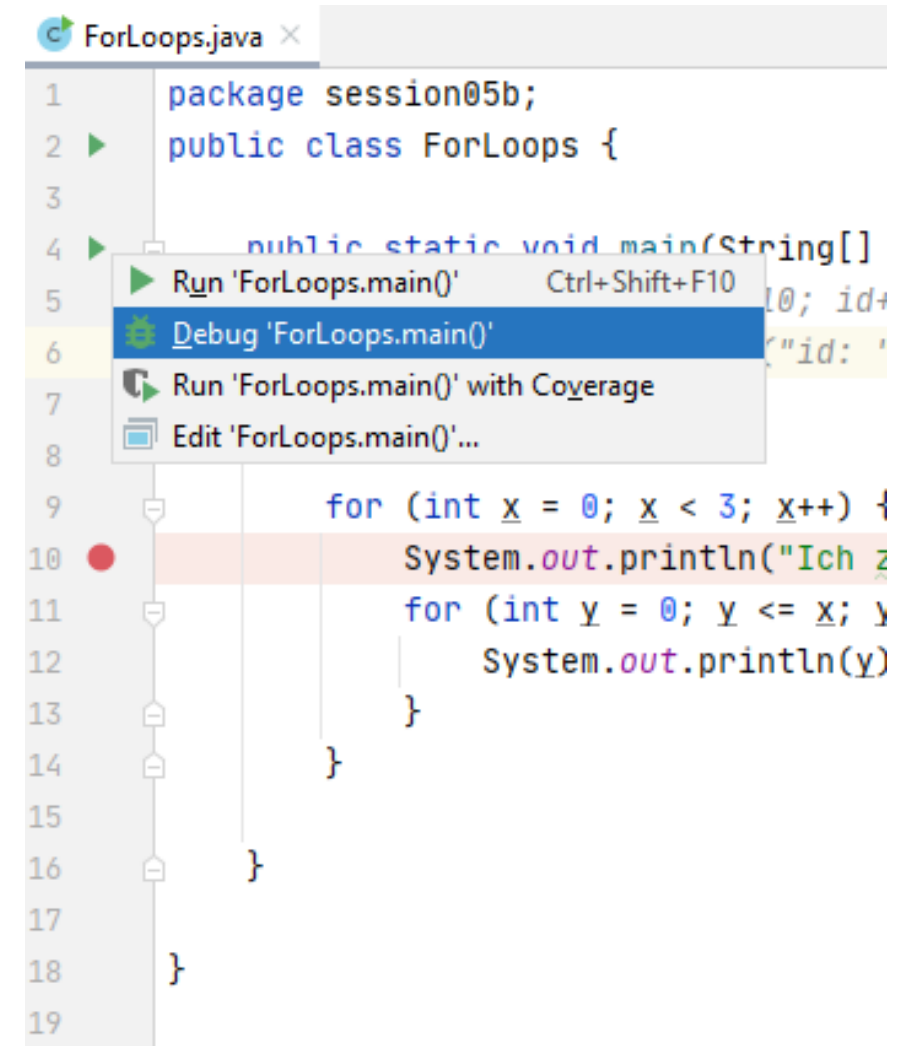
- Einfachste Möglichkeit: Man baut Konsolenausgaben ein (printf oder println)
- Nachteile:
 - Nicht sehr elegant
 - führt zu viel Ausgabe
 - Programm läuft immer weiter
 - man muss dran denken, die Ausgaben wieder zu entfernen
- **Besser: Debugger benutzen!**



code04.
Absolute
Value1
Debug

Debugger und Breakpoints

- Debugging: Finden und Entfernen von "Bugs" (Fehlern) im Programm
- Nachverfolgen des Kontrollflusses
- Breakpoint weist die Laufzeitumgebung an, die Ausführung an dieser Stelle anzuhalten, die Ausführung wird "unterbrochen"
- IntelliJ:
 - Strg + F8 zum Breakpoint Setzen/Entfernen
 - Klick neben main-Methode → Debug oder
 - Debug-Tool-Window
 - F8 Einzelschrittausführung nach Halt
 - F9 normal weiter bis zum nächsten Breakpoint



SlideExamples [...\IntelliJ\SlideExamples] - ForLoops67.java

SlideExamples > src > session05b > ForLoops67

Project

- CharAsInt0
- CharFromInt7
- Declarations1
- ExplicitDowncast5
- FloatUpcastPrecisionLoss4
- ImplicitUpcast3
- MathToIntExact6
- String7
- StringToNumber8
- session04
 - ArithmeticOperators
 - CompareOperators
 - ConditionOperator
 - IncDec
 - LeftAssociation
 - PrintF
 - ShortCircuit
- session05a
 - AbsoluteValue2

ForLoops67.java

```
1 package session05b;
2 public class ForLoops67 {
3
4     public static void main(String[] args) { args: {}
5         //     for (int id = 0; id < 10; id++) {
6         //         System.out.println("id: " + id);
7         //     }
8
9         for (int x = 0; x < 3; x++) { x: 1
10             System.out.println(x);
11             for (int y = 0; y < x; y++) { y: 0
12                 System.out.println("(" + x + "," + y + ")"); x: 1 y: 0
13             }
14         }
15
16     }
17 }
```

Werte direkt im Code

Debugger

Frames

- "main"@1 in g...main: RUNNING
- main:12, ForLoops67 (session05b)

Variables

- x+y = 1
- args = {String[0]@792}
- x = 1
- y = 0

Weiter

Stop

Eigene "Watches" Variablen

Umschalten zur Konsole

Einzelstschritt

4: Run 6: Problems 5: Debug Terminal Build Event Log

All files are up-to-date (a minute ago) 12:1 CRLF UTF-8 Tab*